Ziniu Zheng

Berkeley, CA | ziniuzheng@berkeley.edu | github.com/ZiniuZheng | linkedin.com/in/ziniu-zheng | ziniuzheng.github.io

EDUCATION

University of California, Berkeley **Berkeley, CA**
Bachelor of Science, Electrical Engineering & Computer Sciences Expected Dec 2027
Relevant Coursework: Data Structures, Machine Structures, Algorithms, Artificial Intelligence, Computer Graphics

TECHNICAL SKILLS

Programming Languages: Java, TypeScript, JavaScript, Go, Python, C
Databases and Cloud: PostgreSQL, MySQL, Elasticsearch, AWS (ECS, RDS, ECR), GCP
Web: React, Express, Spring Boot, Node.js, HTML/CSS
Tools & Libraries: Git, Docker, GitHub Actions, LangChain, Spring Security, JUnit, Mockito, Ant Design, Claude Code, Cursor

PROJECTS

ContextLens: AI Browser Assistant (Chrome Extension) *TypeScript, Chrome MV3, OpenAI/Anthropic APIs*
- Built a Chrome extension that brings AI into any web page: select text to explain, summarize, or rewrite it in an in-place floating panel, with answers streaming token by token from OpenAI or Anthropic models
- Wrote an incremental Server-Sent Events parser that handles events split across network chunks; closing the panel disconnects the port and aborts the in-flight request
- Isolated the overlay in a closed Shadow DOM built with native DOM APIs and rendered model output as text only; kept API keys in the background service worker with host permissions limited to the two model APIs
- Shipped with zero runtime dependencies, 26 unit tests, and GitHub Actions CI (type-check, test, build) per push

Agent AI: Full-Stack RAG & MCP Document Q&A Agent *React, Node.js, Express, LangChain, GPT-5, MCP*
- Developed a full-stack AI agent using React and Ant Design, with voice interaction powered by react-speech-recognition and speak-tts
- Implemented a Retrieval-Augmented Generation (RAG) pipeline leveraging GPT-5 and LangChain components to enable semantic document comprehension and context-aware responses
- Designed a hybrid search strategy combining RAG outputs with web search through a Model Context Protocol (MCP) server, standardizing tool access and integrating results from SerpAPI
- Built and deployed a backend service with Node.js and Express, supporting document ingestion, query handling, and API orchestration

OnlineOrder: Online Food Ordering Web App *Spring Boot, PostgreSQL, React, Docker, AWS, JUnit*
- Developed RESTful APIs with Spring Boot covering registration, menu browsing, cart management, and checkout, structured into controller, service, and repository layers with dependency injection
- Leveraged Spring Data JDBC with PostgreSQL on AWS RDS to manage restaurants, menus, and carts; added Caffeine caching with eviction on cart updates
- Secured endpoints with Spring Security session-based authentication and BCrypt password hashing
- Wrote 27 JUnit 5 / Mockito unit tests for the service layer (ordering, checkout, sign-up), including Spring cache tests verifying @Cacheable/@CacheEvict invalidation
- Containerized the app with a multi-stage Docker build and set up GitHub Actions CI that runs backend and frontend tests and smoke-tests the image on every push
- Deployed to AWS ECS Fargate behind an HTTPS load balancer with a private RDS database; automated deployment and teardown with AWS CLI scripts

SocialAI: AI-Powered Social Network *React, Go, Elasticsearch, DALL-E 3, GCP*
- Built a social network web application with React
- Integrated OpenAI's DALL-E 3 to help users create and update posts
- Implemented token-based registration/login/logout flows with React Router and server-side JWT authentication
- Launched a scalable Go web service to handle posts and deployed it to Google App Engine
- Used Elasticsearch on Google Compute Engine (GCE) to let users search and browse recent posts